

```
include <stdio.h>

line SUCCESS 0

somme(int e1, int e2)

return (e1 + e2);

main(int argc, char con

int a=4, b=5;
printf("%d\n", som
return SUCCESS;
```

Langage C

M. DIOUF

Université Amadou Mahtar MBOW

Licence 2 Tronc Commun - Polytech Diamniadio

Version 3 du cours



Présentation Générale

Destinataire

- Polytech Diamniadio / UAM

Intitulé du Cours

- 2INF1303 - Programmation orientée système (Langage C)

Charge horaire, CM – TD/TP

Présentation Générale

Public cible:

- Licence 2 Tronc Commun

Pré – requis

- Introduction à l'algorithmique
- Notions en programmation
- Notions en architecture des ordinateurs

Objectifs généraux du cours

Maîtriser les fondements de la programmation en langage C

Concevoir des programmes modulaires à l'aide de fonctions

Manipuler et organiser les données à l'aide des structures adaptées

Objectifs spécifiques

Maîtriser la syntaxe de base du langage C.

Déclarer et manipuler des variables de différents types, utiliser les opérateurs et les expressions.

Mettre en œuvre les structures conditionnelles et les structures de contrôle.

Réaliser des opérations d'entrée/sortie.

Développer des algorithmes utilisant les boucles et les mécanismes d'itération.

Comprendre les principes de la programmation procédurale.

Décomposer un problème complexe en sous-problèmes.

Concevoir et utiliser des fonctions en langage C.

Organiser un programme en modules réutilisables et maintenables.

Maîtriser le passage de paramètres et les valeurs de retour.

Manipuler des collections de données à l'aide des tableaux.

Traiter efficacement des ensembles d'informations homogènes.

Définir et utiliser des enregistrements (struct) pour représenter des données complexes.

Combiner tableaux et enregistrements pour modéliser des problèmes réels.

Développer des programmes utilisant des structures de données adaptées

Formules pédagogiques

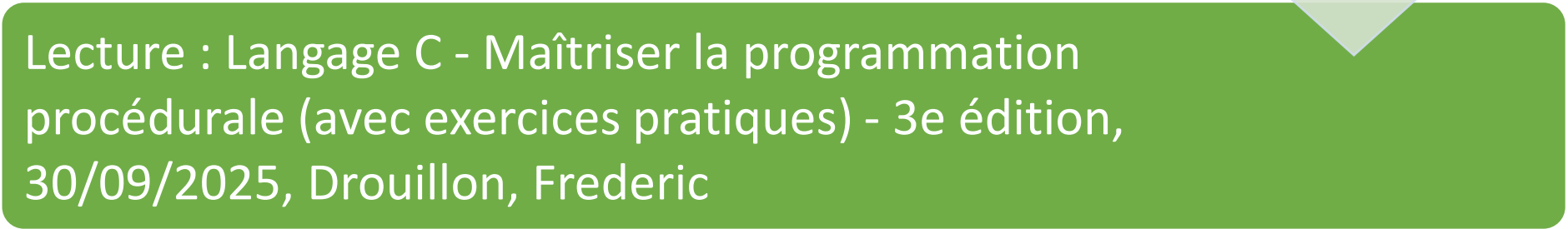
Exposé de cours magistraux



Travaux dirigés et pratiques



Lecture : Langage C - Maîtriser la programmation
procédurale (avec exercices pratiques) - 3e édition,
30/09/2025, Drouillon, Frederic



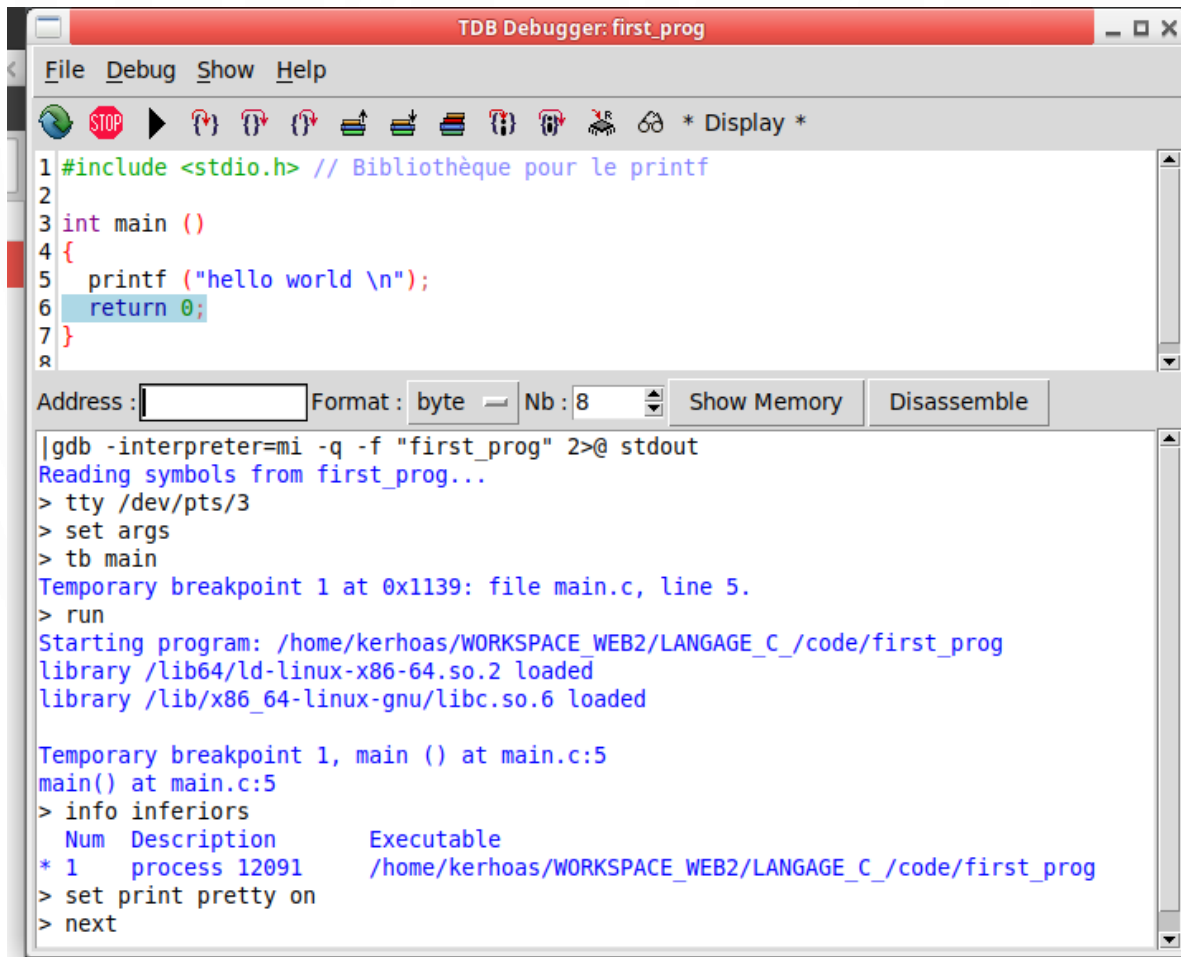
Evaluation



Devoirs écrits (40 %)



Examen écrit (60 %)



The screenshot shows the TDB Debugger window titled "TDB Debugger: first_prog". The main pane displays a C program with the following code:

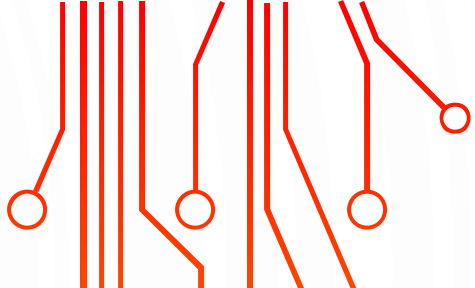
```
1 #include <stdio.h> // Bibliothèque pour le printf
2
3 int main ()
4 {
5     printf ("hello world \n");
6     return 0;
7 }
8
```

Below the code, the "Address" field is empty, and the "Format" is set to "byte". The "Nb" field is set to 8. The "Show Memory" and "Disassemble" buttons are visible.

The bottom pane shows the GDB console output:

```
[gdb -interpreter=mi -q -f "first_prog" 2>@ stdout
Reading symbols from first_prog...
> tty /dev/pts/3
> set args
> tb main
Temporary breakpoint 1 at 0x1139: file main.c, line 5.
> run
Starting program: /home/kerhoas/WORKSPACE_WEB2/LANGAGE_C/code/first_prog
library /lib64/ld-linux-x86-64.so.2 loaded
library /lib/x86_64-linux-gnu/libc.so.6 loaded

Temporary breakpoint 1, main () at main.c:5
main() at main.c:5
> info inferiors
  Num  Description      Executable
* 1    process 12091    /home/kerhoas/WORKSPACE_WEB2/LANGAGE_C/code/first_prog
> set print pretty on
> next
```

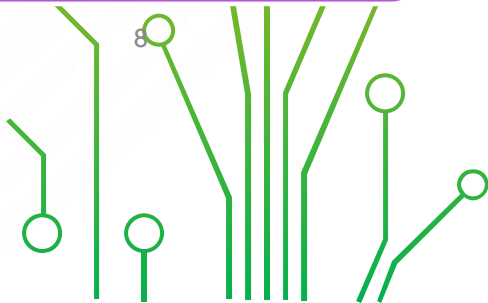


Chapitre 1 : Introduction au langage C

Chapitre 2 : Sous - programmes

Chapitre 3 : Tableau et enregistrement

Chapitre 4 : Fichiers



Révision Algo

Exercice 1 : (25 min)

Écrire un algorithme qui permet de saisir une série de valeurs ensuite renvoie :

- La somme des entiers positifs
- La somme des entiers pairs
- La somme des entiers premiers

Exercice 2 : (25 min)

Écrire un algorithme qui permet de saisir les notes de n étudiants puis affiche :

- la moyenne générale
- la plus grande note
- la plus petite note
- le nombre d'étudiants ayant la moyenne

Exercice 3 : (25 min)

Écrire un algorithme qui permet de calculer la factorielle d'un entier

Exercice 4 : (25 min)

Écrire un algorithme qui permet de calculer la puissance d'un entier sans utiliser l'exposant



Chapitre 1

Introduction au langage C

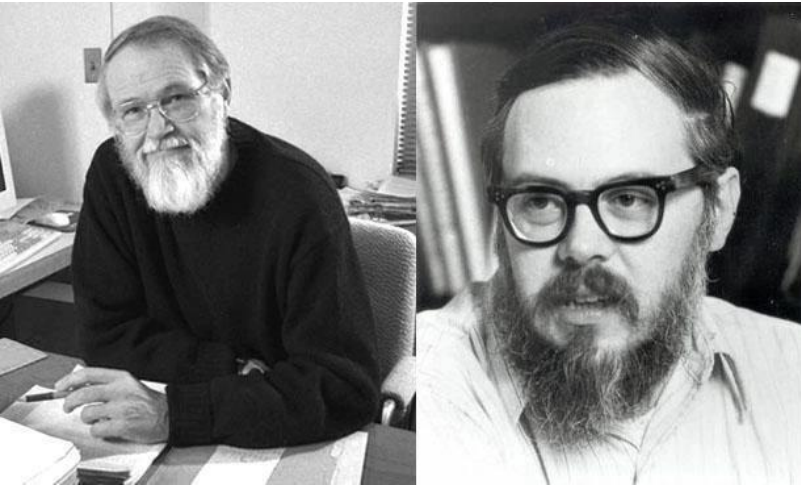
Préambules

Éléments de base

Structures de contrôle

Préambules

Un peu d'histoire



- Développé dans les années 1970 par Kernighan et Ritchie aux laboratoires Bell d'AT&T, le C a été conçu pour être à la fois **simple** et **puissant** ;
- Il a été conçu pour réécrire en langage évolué le 1er **système** d'exploitation UNIX de manière à assurer sa portabilité ;
- Il est très plébiscité pour développer des **systèmes** d'exploitation, des langages de programmation et pour écrire aussi bien des applications de calcul scientifique ;
- C'est le 1er langage à être quasiment disponible sur tout **système** programmable (PC, station, microcontrôleur...) ;
- Syntaxe utilisée par plusieurs langages de programmation comme le Java ;
- Extension au C++ en 1985 qui permet la POO ;
- Le langage C est le pilier de la programmation pour les **systèmes** embarqués et constitue le langage standard pour Arduino, tout en étant très utilisé sur Raspberry Pi.

Préambules

Caractéristiques du Langage C

C'est un langage très utilisé dans l'industrie car il cumule les avantages d'un langage de haut-niveau (portabilité, modularité, etc...) et ceux des langages assembleurs proches du matériel.

- **Langage de haut niveau**

Programmation structurée (conçu pour traiter les tâches d'un programme en les mettant dans des blocs) ;

Programmation sous forme de fonctions (sous-programmes).

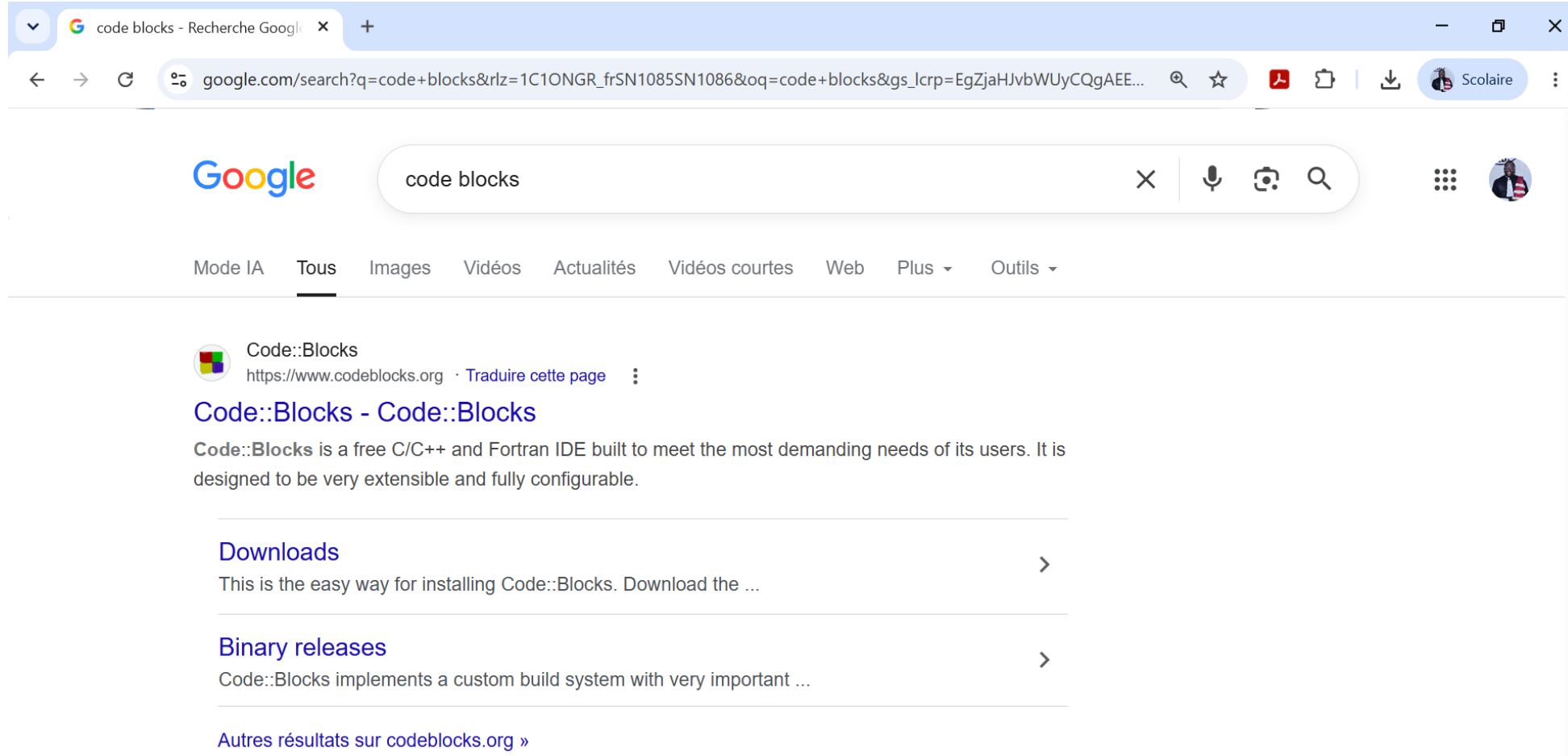
- **Langage de bas-niveau (niveau matériel)**

Conçu pour être facilement traduit en langage machine ;

Gestion de la mémoire "à la main".

Préambules

Préparation de l'environnement de développement intégré



Préambules

Préparation de l'environnement de développement intégré

features you need,
having a consistent
look, feel and
operation across
platforms.

News
Downloads
User manual
Forums
Wiki
License
Donations

Code::Blocks

The free C/C++ and Fortran IDE.

Code::Blocks is a free C/C++ and Fortran IDE built to meet the most demanding needs of its users. It is designed to be very extensible and fully configurable.

Built around a plugin framework, Code::Blocks can be extended with plugins. Any kind of functionality can be added by installing/coding a plugin. For instance, event compiling and debugging functionality is provided by plugins!

If you're new here, you can read the [user manual](#) or visit the [Wiki](#) for documentation. And don't forget to visit and join our [forums](#) to find help or general discussion about Code::Blocks.

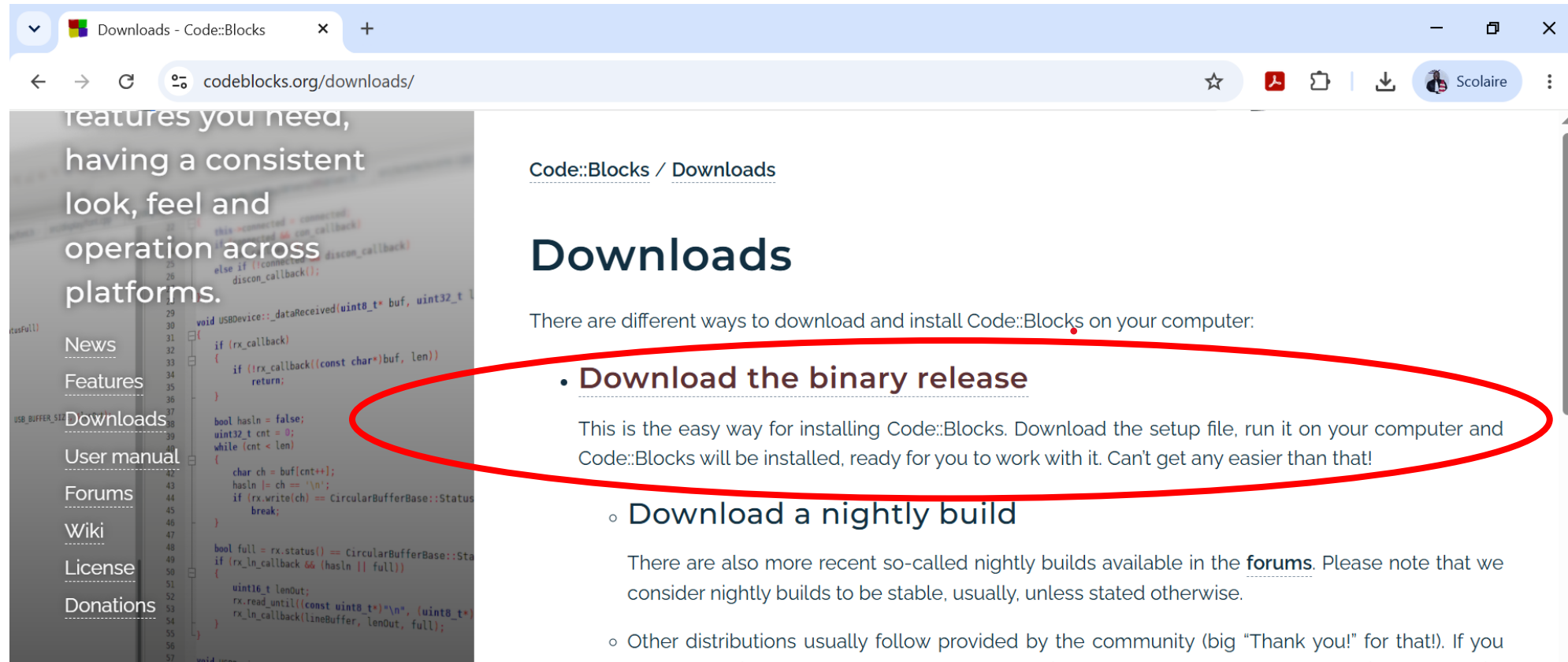
We hope you enjoy using Code::Blocks!

The Code::Blocks Team

Latest news

Préambules

Préparation de l'environnement de développement intégré



The screenshot shows a web browser window with the address bar displaying 'codeblocks.org/downloads/'. The page title is 'Code::Blocks / Downloads'. The main heading is 'Downloads'. Below it, a paragraph states: 'There are different ways to download and install Code::Blocks on your computer:'. A red circle highlights the first bullet point: '• **Download the binary release**'. Below this, a paragraph explains: 'This is the easy way for installing Code::Blocks. Download the setup file, run it on your computer and Code::Blocks will be installed, ready for you to work with it. Can't get any easier than that!'. Below this paragraph is another bullet point: '• **Download a nightly build**'. Below this, a paragraph states: 'There are also more recent so-called nightly builds available in the [forums](#). Please note that we consider nightly builds to be stable, usually, unless stated otherwise.' At the bottom, a paragraph starts with '• Other distributions usually follow provided by the community (big "Thank you!" for that!). If you'. On the left side of the page, there is a sidebar with a list of links: 'News', 'Features', 'Downloads', 'User manual', 'Forums', 'Wiki', 'License', and 'Donations'. The 'Downloads' link is highlighted. In the background, there is a blurred image of a code editor showing C code.

Code::Blocks / Downloads

Downloads

There are different ways to download and install Code::Blocks on your computer:

- **Download the binary release**

This is the easy way for installing Code::Blocks. Download the setup file, run it on your computer and Code::Blocks will be installed, ready for you to work with it. Can't get any easier than that!
- **Download a nightly build**

There are also more recent so-called nightly builds available in the [forums](#). Please note that we consider nightly builds to be stable, usually, unless stated otherwise.
- Other distributions usually follow provided by the community (big "Thank you!" for that!). If you

Préambules

Préparation de l'environnement de développement intégré



features you need,
having a consistent
look, feel and
operation across
platforms.

News
Features
Downloads
User manual
Forums
Wiki
License
Donations

```
void USBDevice::dataReceived(uint8_t* buf, uint32_t len)
{
    if (rx_callback)
    {
        if (rx_callback((const char*)buf, len))
            return;
    }

    bool hasln = false;
    uint32_t cnt = 0;
    while (cnt < len)
    {
        char ch = buf[cnt++];
        hasln |= ch == '\n';
        if (rx_write(ch) == CircularBufferBase::StatusFull)
            break;
    }

    bool full = rx.status() == CircularBufferBase::StatusFull;
    if (rx_callback && (hasln || full))
    {
        uint16_t lenOut;
        rx.read_until((const uint8_t*)"\\n", (uint8_t*)buf, lenOut, full);
        rx_callback(buf, lenOut, full);
    }
}

void USBDevice::setConnectCallback(CONNECT_CB callback)
{
    con_callback = callback;
}
```

Code::Blocks / Downloads / Binary releases

Binary releases

Please select a setup package depending on your platform:

- [Windows XP / Vista / 7 / 8.x / 10](#)
- [Linux 32 and 64-bit](#)
- [Mac OS X](#)

NOTE: For older OS'es use older releases. There are releases for many OS version and platforms on the [Sourceforge.net](#) or [dappcdn.com](#) page.

NOTE: There are also more recent nightly builds available in the [forums](#) or (for Ubuntu users) in the [Ubuntu PPA repository](#). Please note that we consider nightly builds to be stable, usually.

NOTE: We have a [Changelog for 25.03](#), that gives you an overview over the enhancements and fixes we have put in the new release.

NOTE: The default builds are 64 bit (starting with release 20.03). We also provide 32bit builds only for convenience.

GPLv3 W3C CSS GetFirefox

Préambules

Préparation de l'environnement de développement intégré

having a consistent look, feel and operation across platforms.

News
Features
Downloads
User manual
Forums
Wiki
License
Donations

NOTE: The default builds are 64 bit (starting with release 25.03) for convenience.

Microsoft Windows (64-bit)

File	Download
codeblocks-25.03-setup.exe	dAppCDN.com or Sourceforge.net
codeblocks-25.03-setup-nonadmin.exe	dAppCDN.com or Sourceforge.net
codeblocks-25.03-nosetup.zip	dAppCDN.com or Sourceforge.net
codeblocks-25.03mingw-setup.exe	dAppCDN.com or Sourceforge.net
codeblocks-25.03mingw-nosetup.zip	dAppCDN.com or Sourceforge.net

NOTE: The codeblocks-25.03-setup.exe file includes Code::Blocks with all plugins. The codeblocks-25.03-setup-nonadmin.exe file is provided for convenience to users that do not have administrator rights on their machine(s).

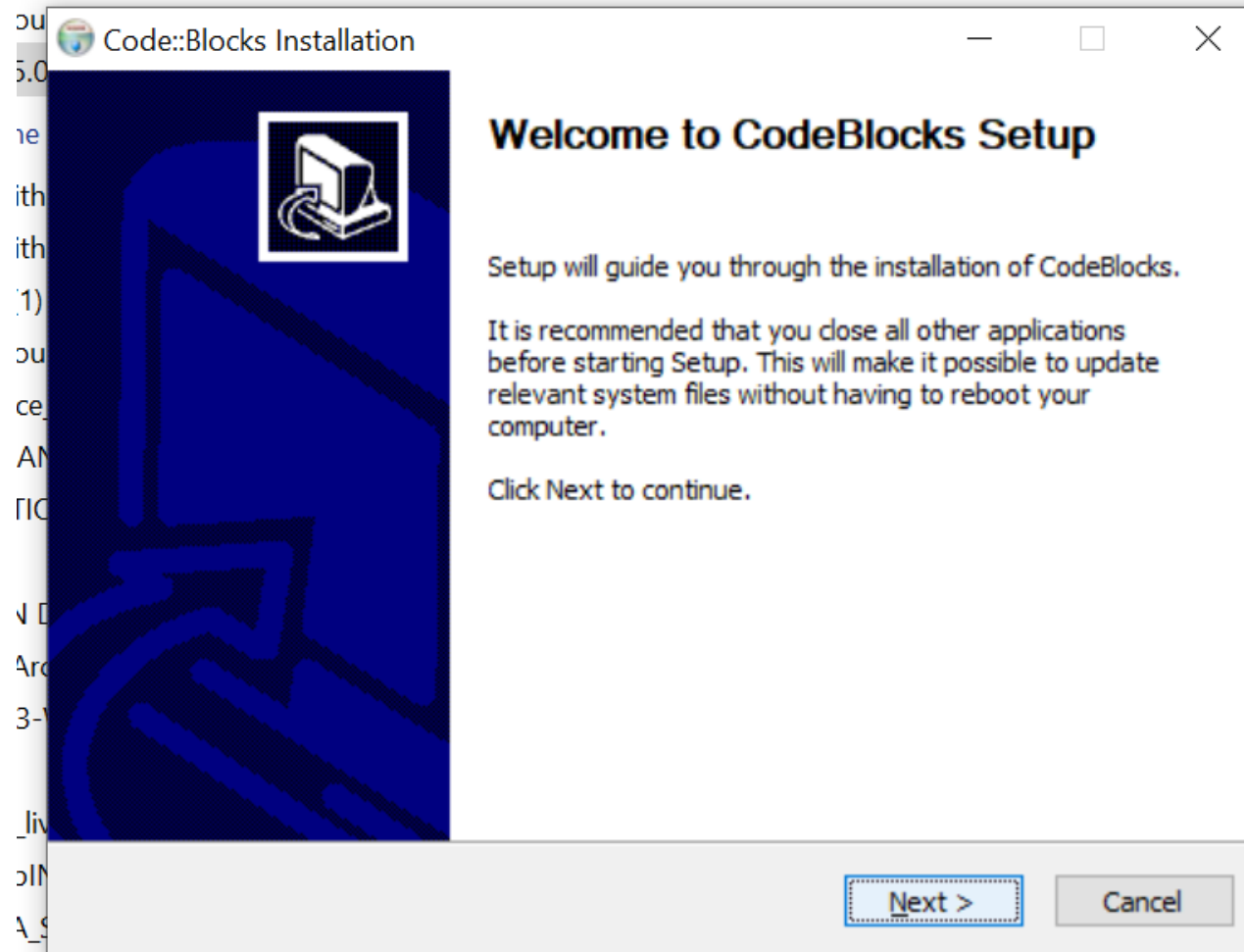
Historique des téléchargements récents

- codeblocks-25.03mingw-setup.exe
↓ 15,0/404 Mo • 4 minutes restantes
- codeblocks-25.03-setup.exe
Supprimé

Historique complet des téléchargements

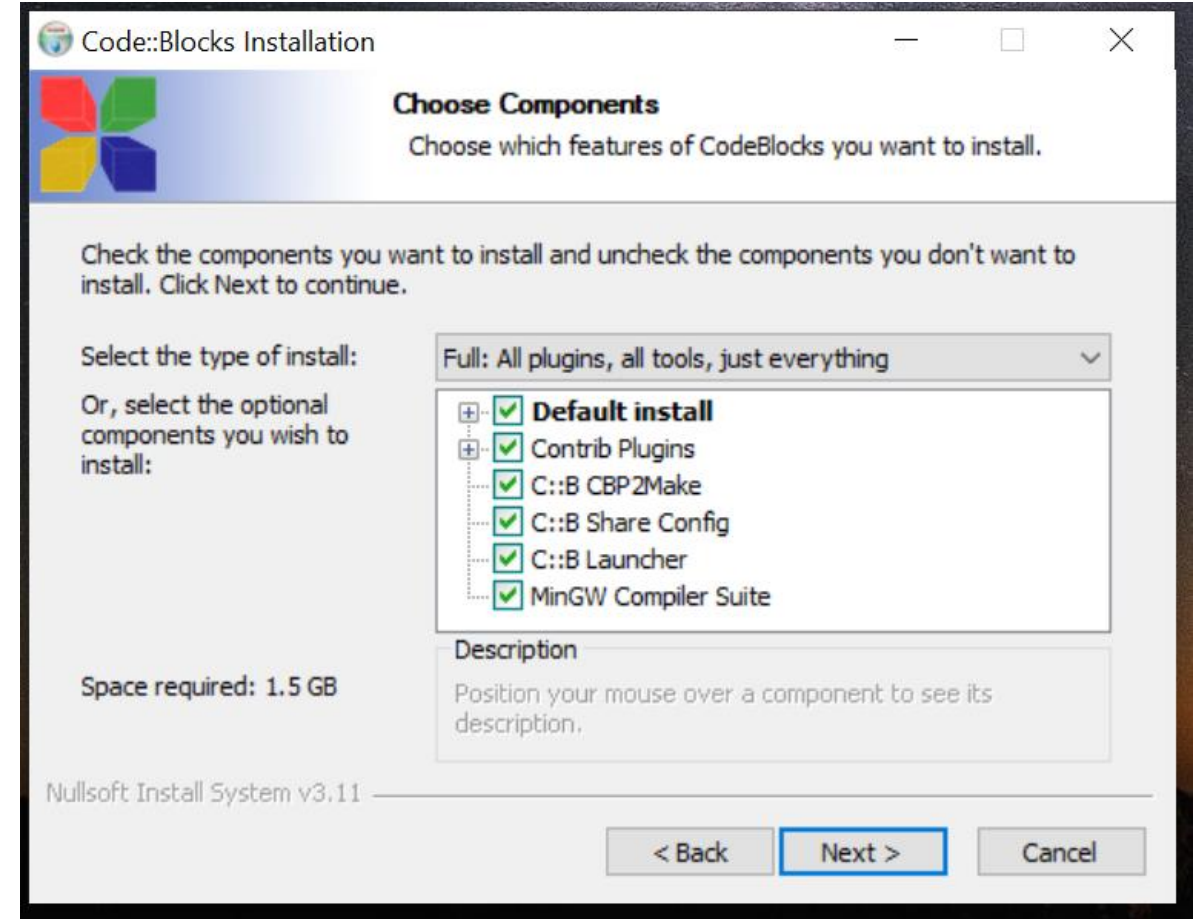
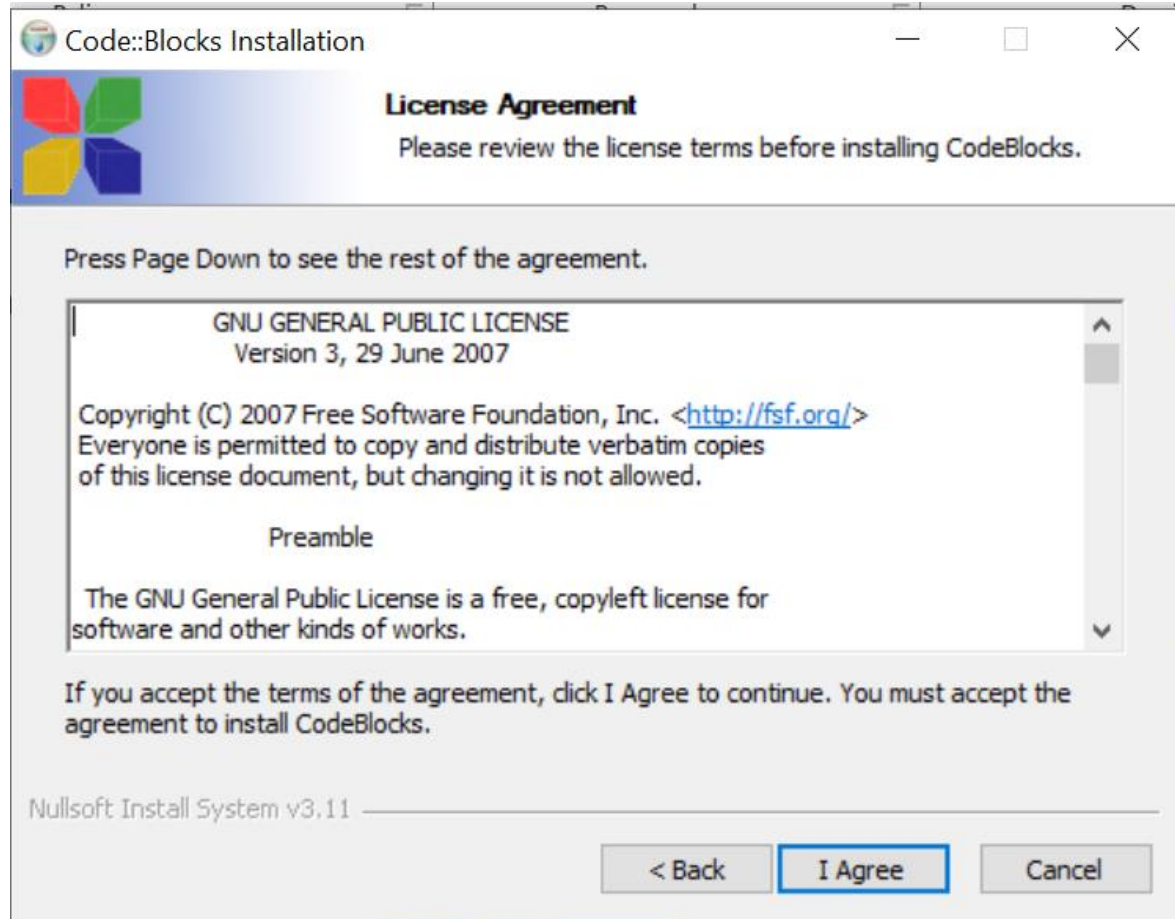
Préambules

Préparation de l'environnement de développement intégré



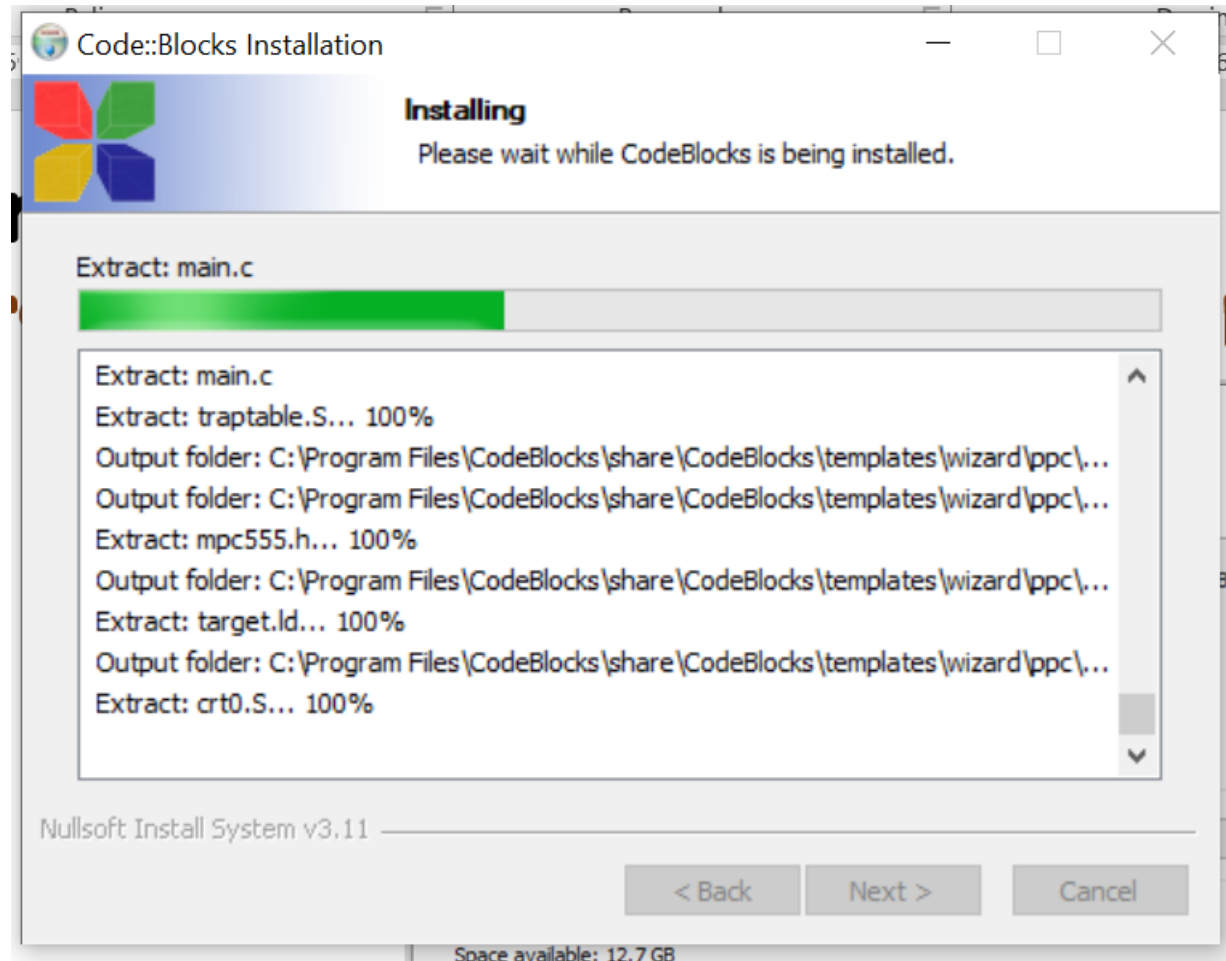
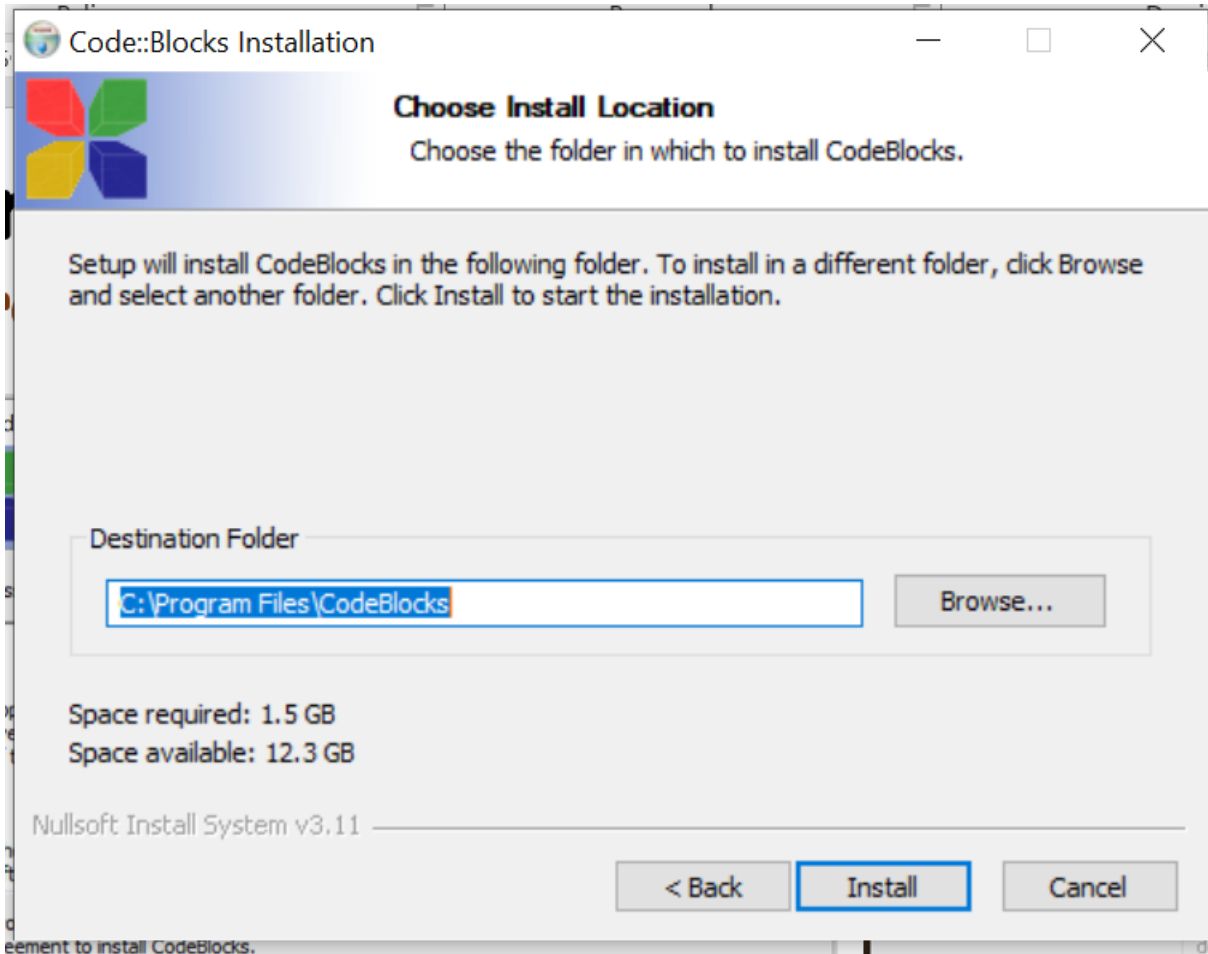
Préambules

Préparation de l'environnement de développement intégré



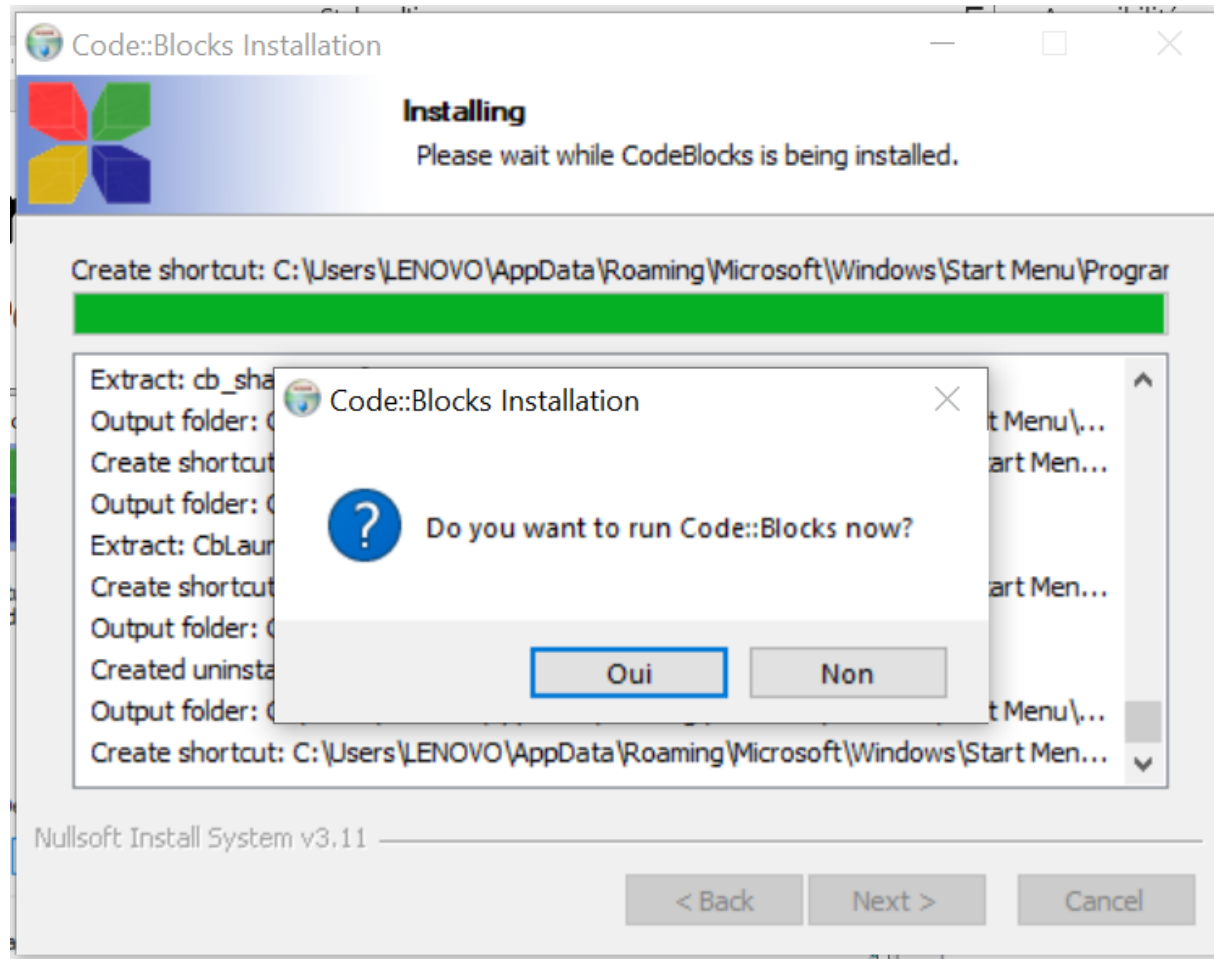
Préambules

Préparation de l'environnement de développement intégré



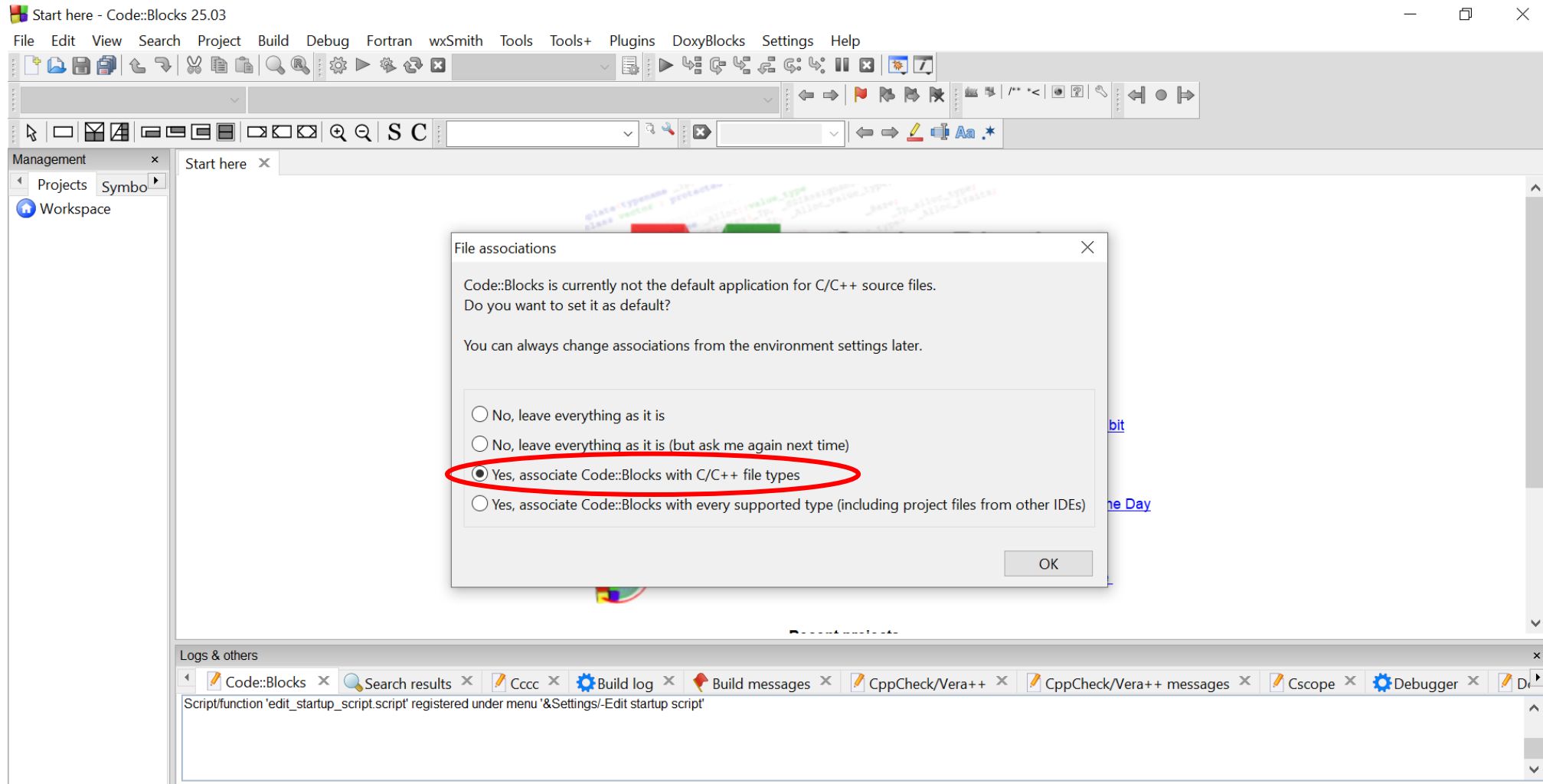
Préambules

Préparation de l'environnement de développement intégré



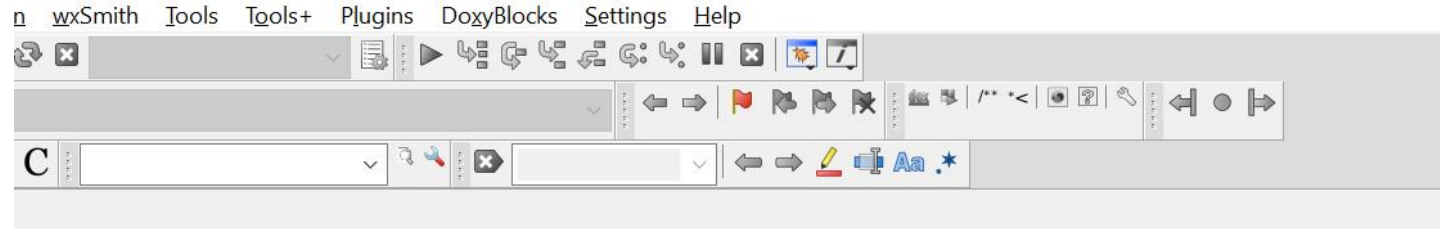
Préambules

Préparation de l'environnement de développement intégré



Préambules

Préparation de l'environnement de développement intégré



[Release 25.03 rev 13644 \(2025-03-29 05:36:19\) gcc 14.2.0 Windows/unicode - 64 bit](#)



[Create a new project](#)



[Open an existing project](#)



[Tip of the Day](#)

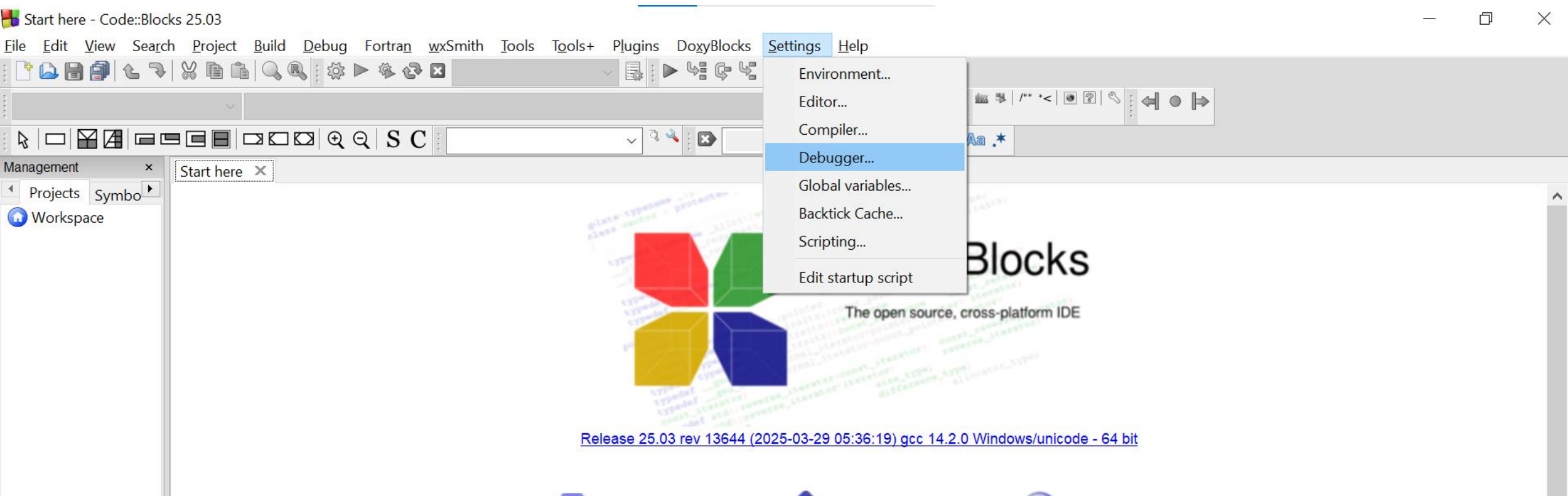


[Visit the Code::Blocks forums](#)

[Report a bug or request a new feature](#)

Préambules

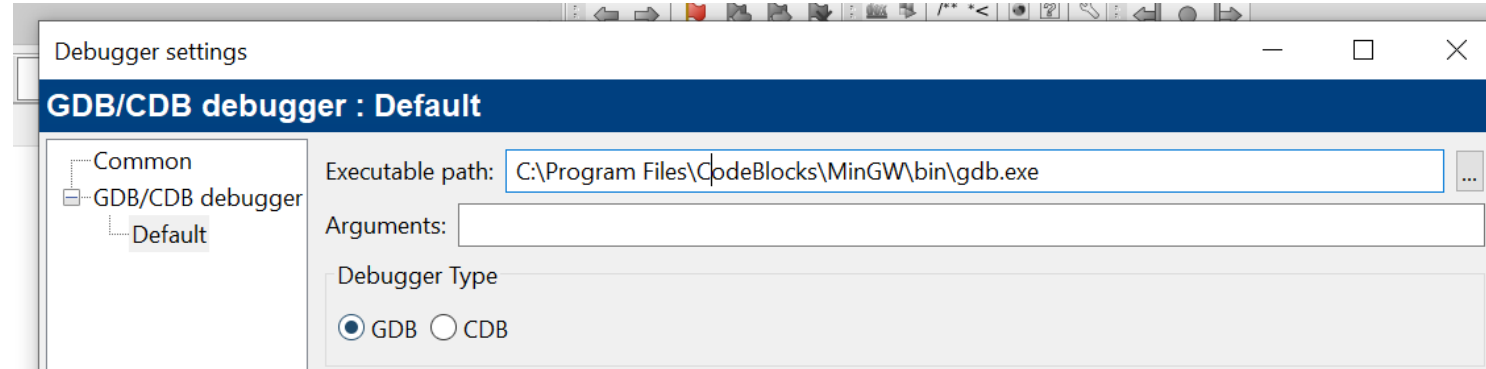
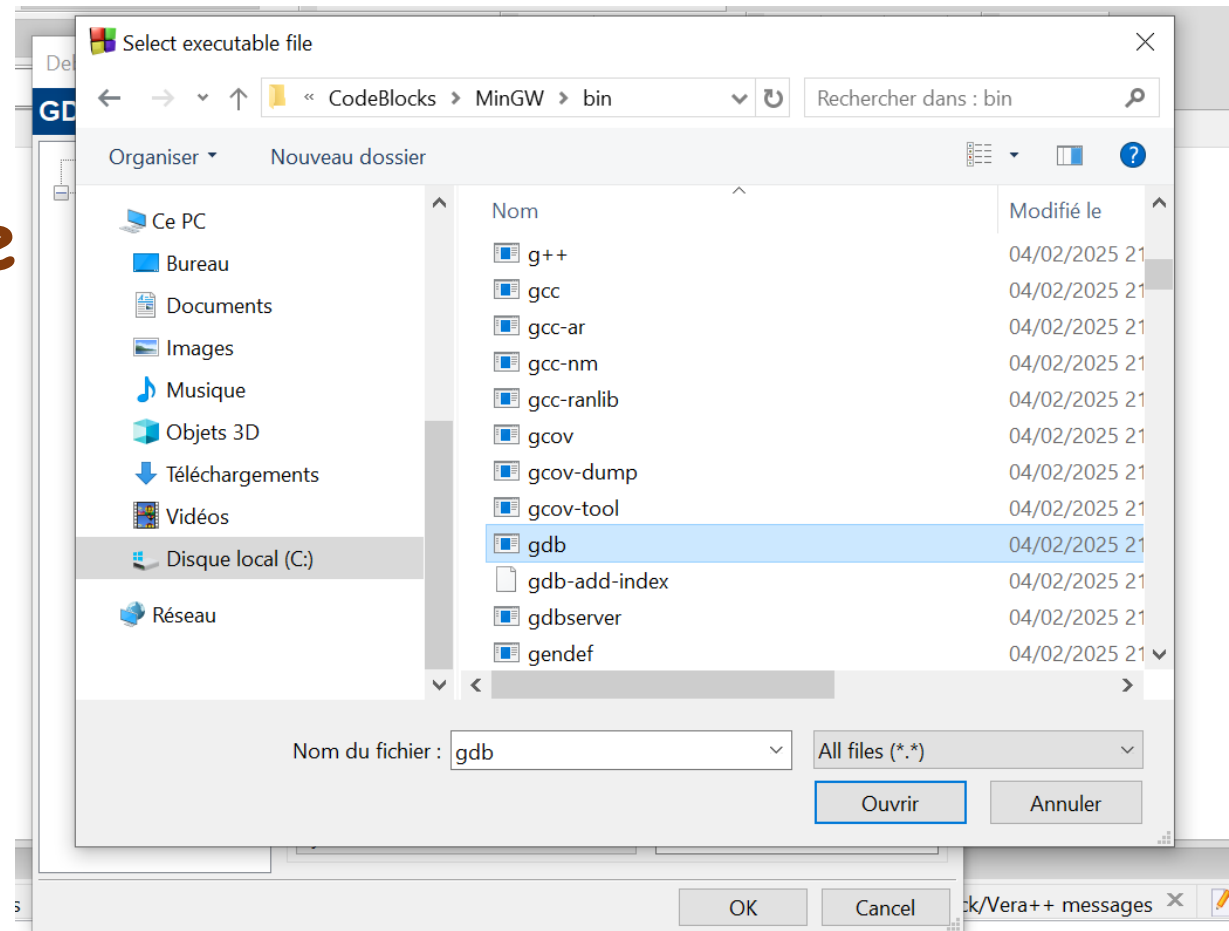
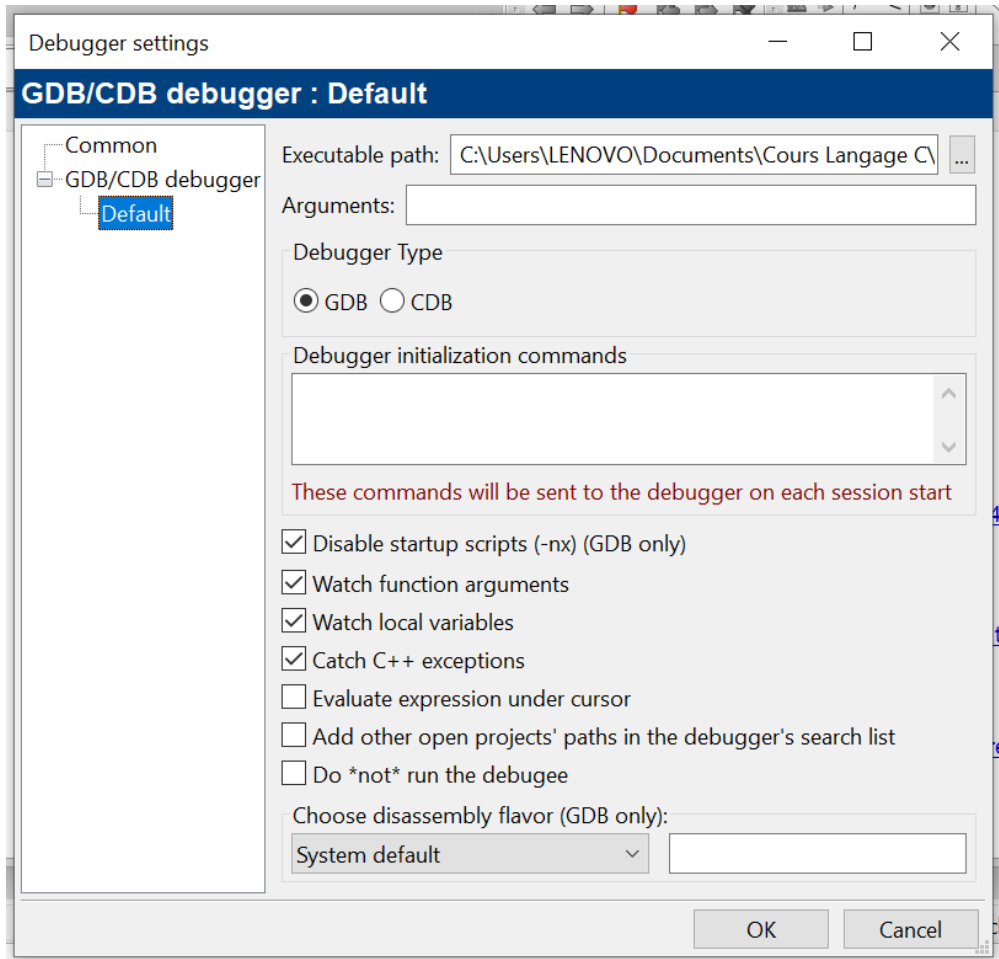
Préparation de l'environnement de développement intégré - Pose du debugger



Préambules

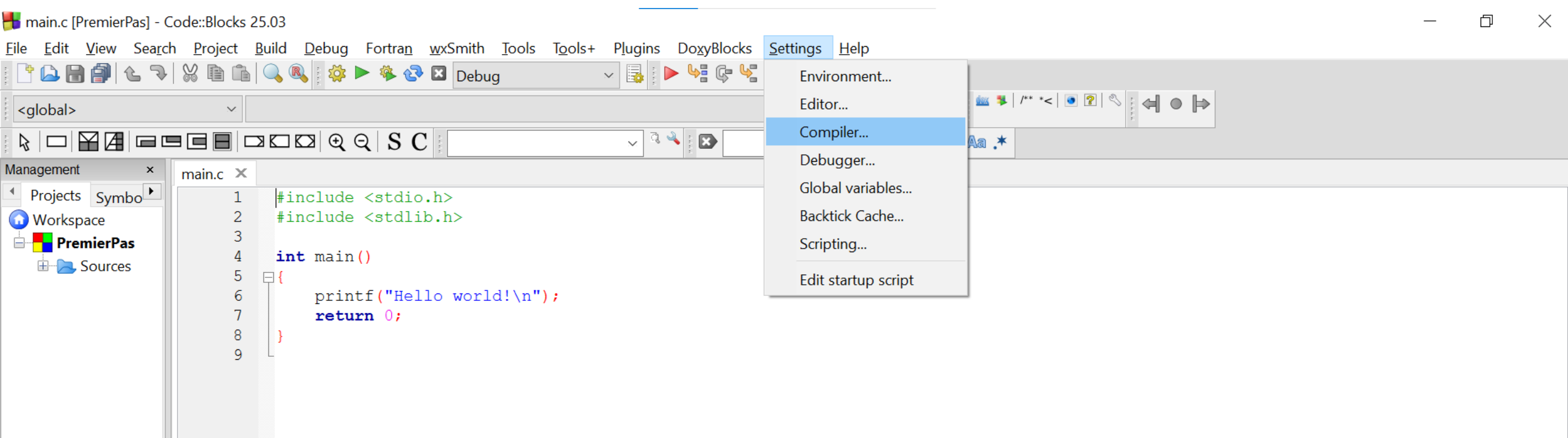
Préparation de l'environne

é



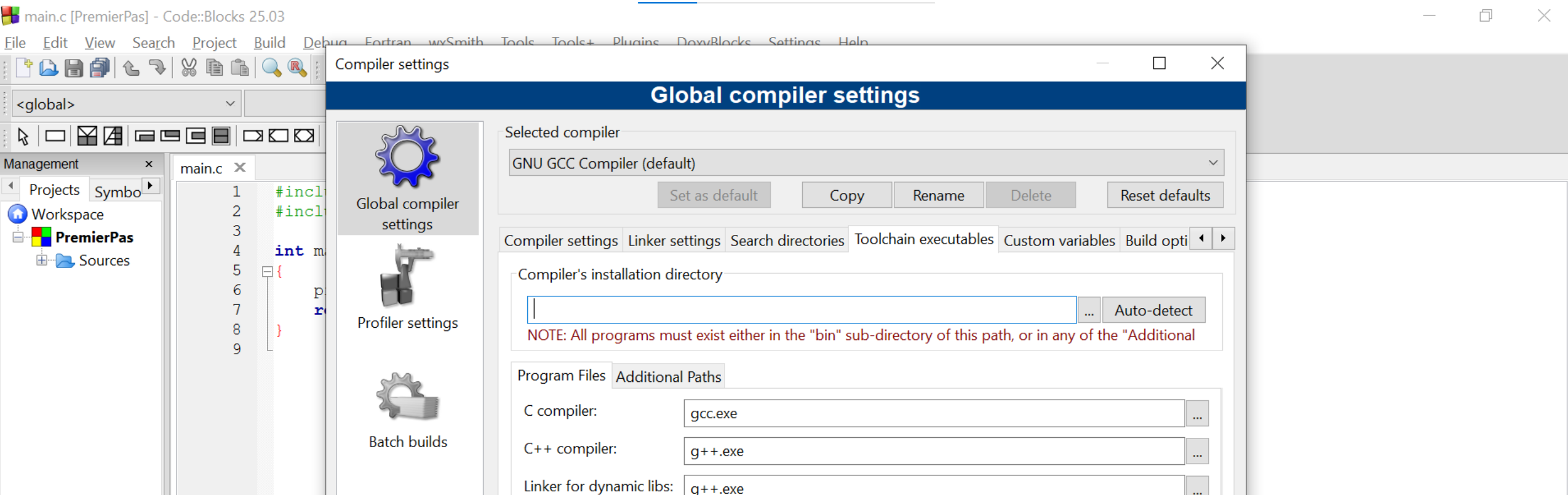
Préambules

Préparation de l'environnement de développement intégré - Pose du compiler



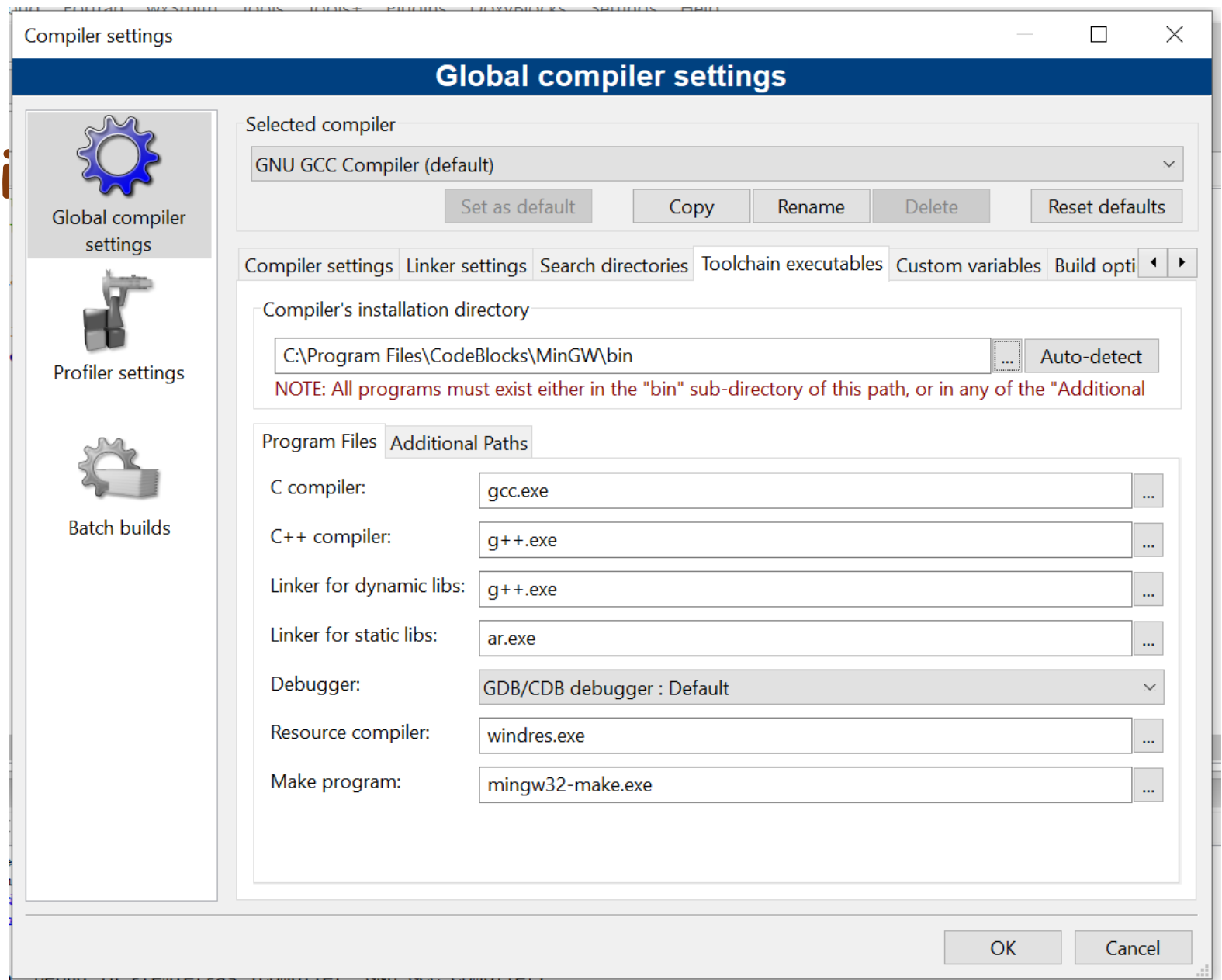
Préambules

Préparation de l'environnement de développement intégré - Pose du compiler



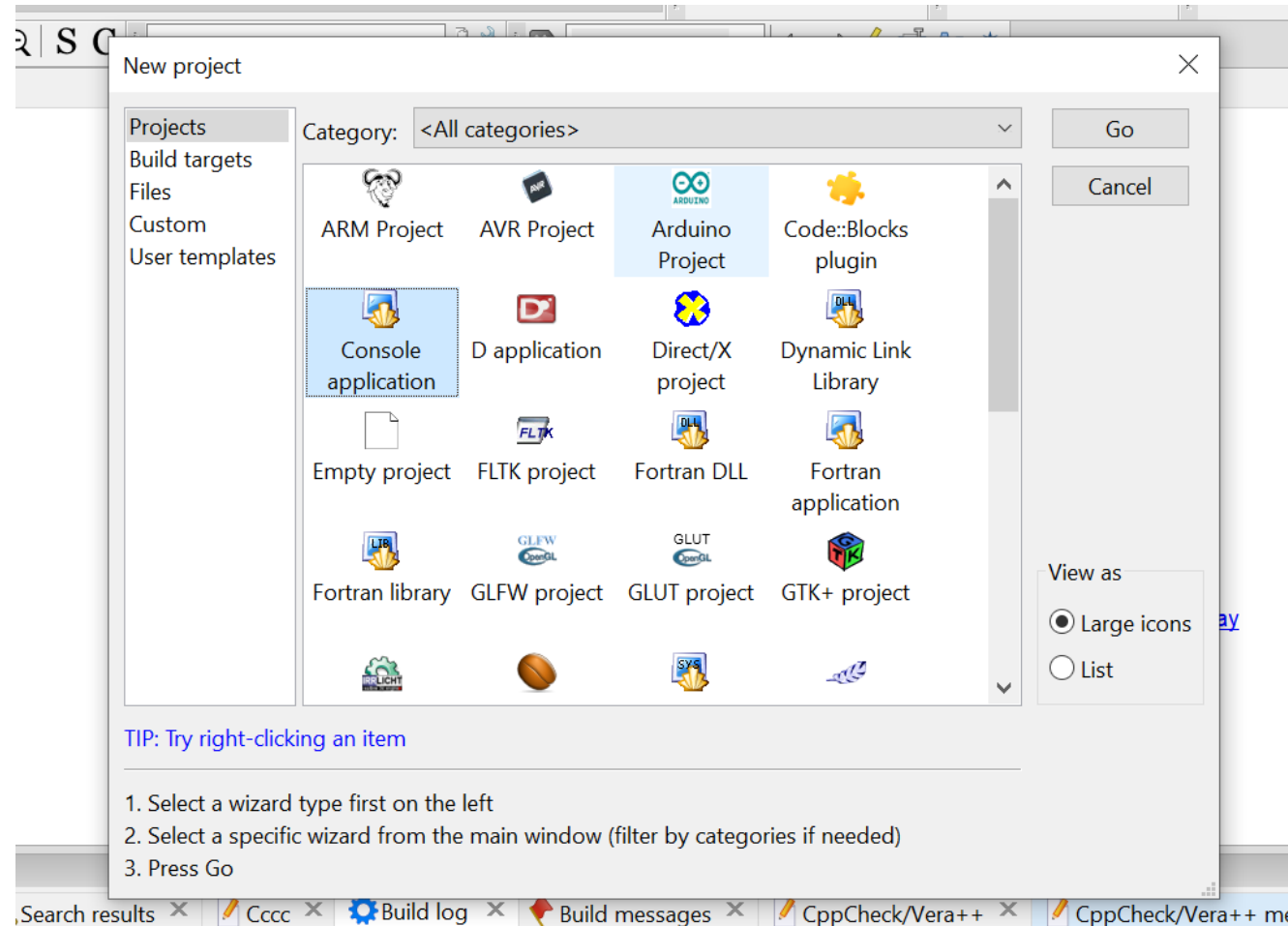
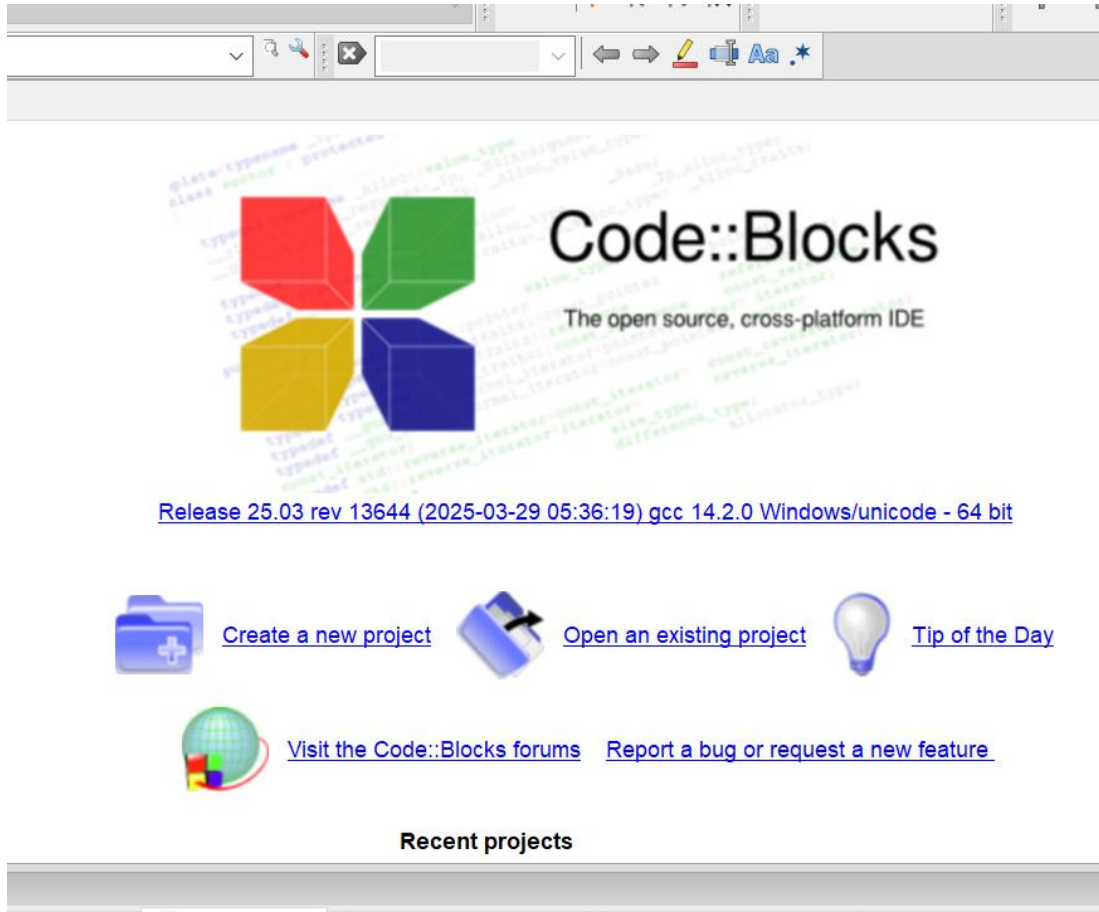
Préambules

Préparation de l'environnement - Pose du compiler



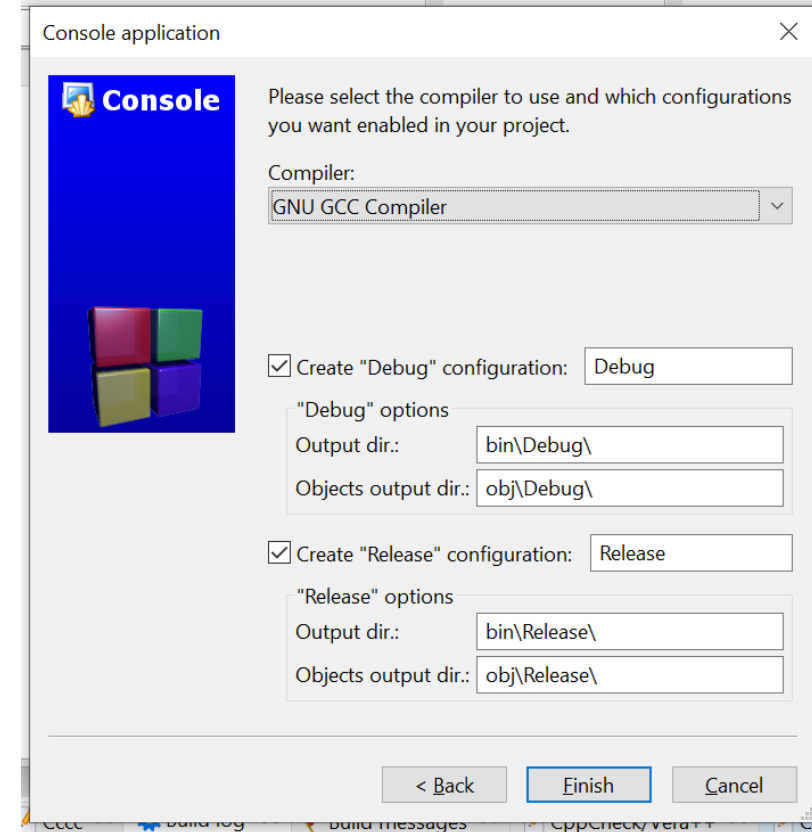
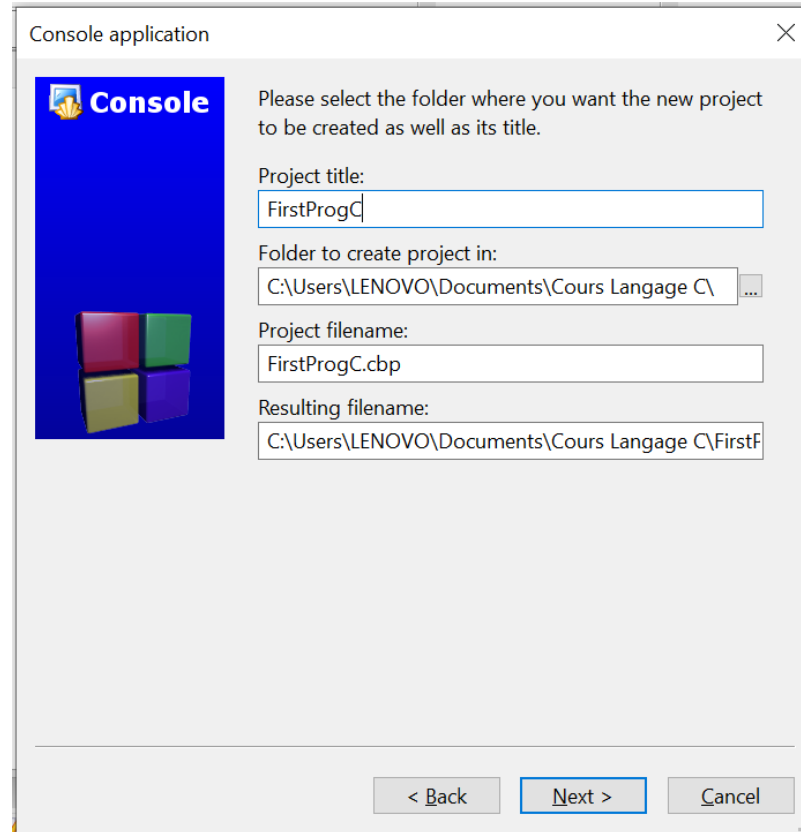
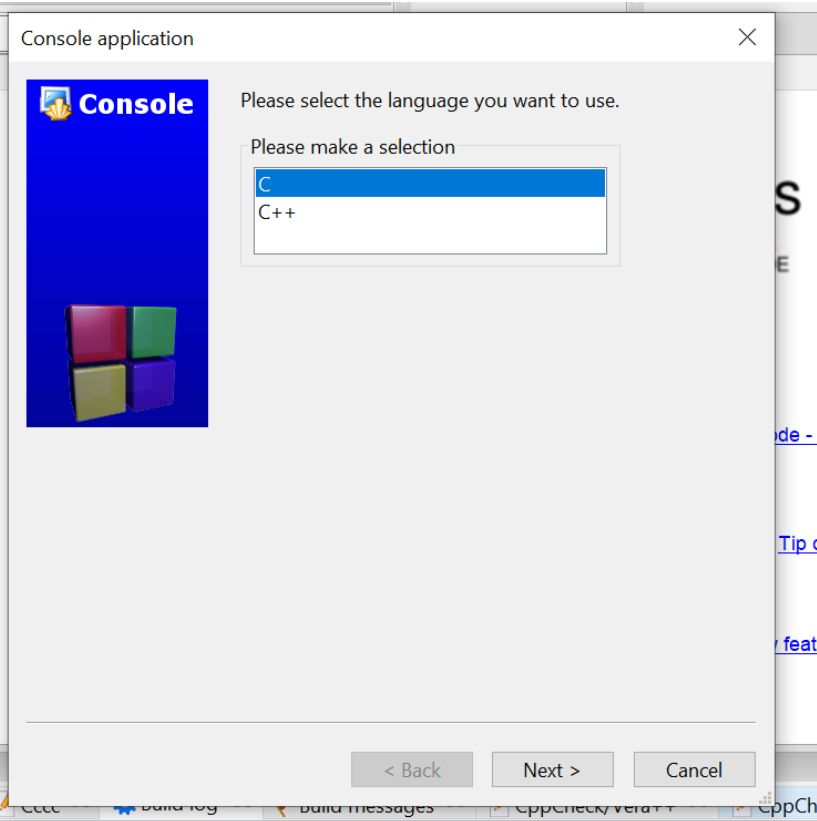
Préambules

Création d'un premier programme



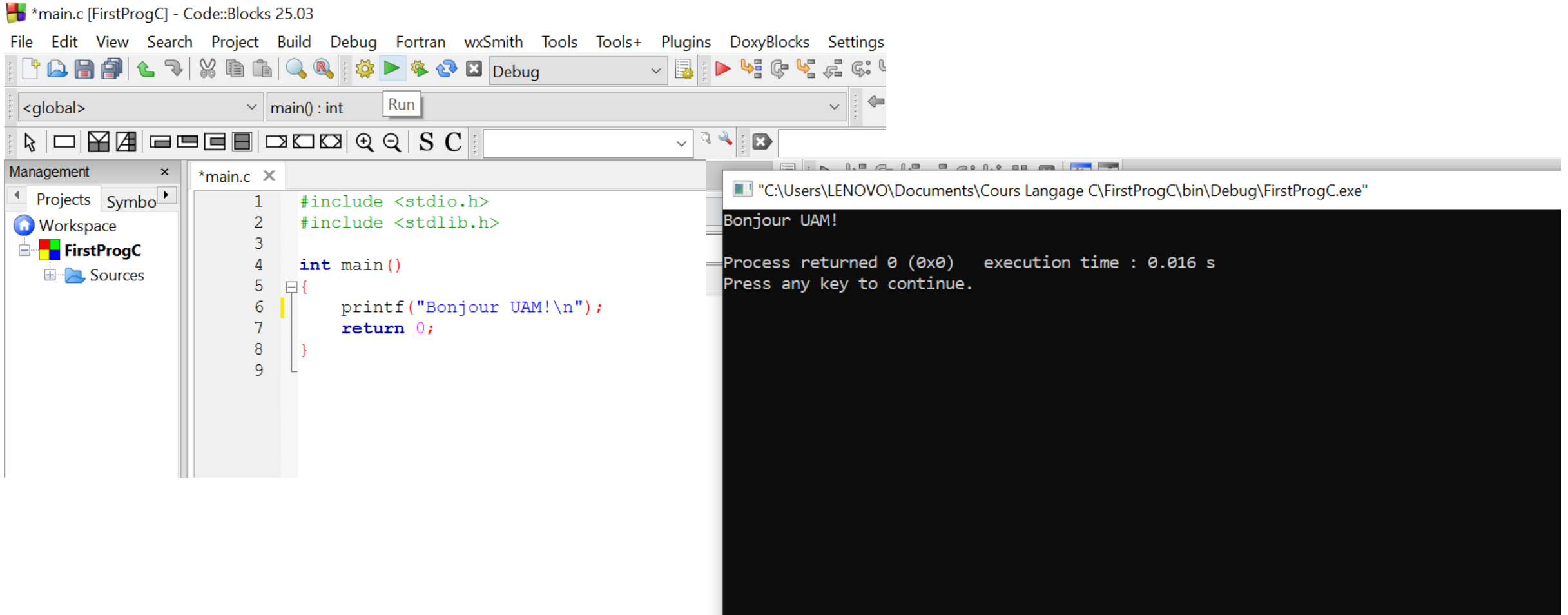
Préambules

Création d'un premier programme



Préambules

Création d'un premier programme



The screenshot displays the Code::Blocks IDE interface. The main editor window shows a C program named `*main.c` with the following code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     printf("Bonjour UAM!\n");
7     return 0;
8 }
9
```

The IDE's toolbar includes a `Run` button. A console window on the right shows the output of the program:

```
"C:\Users\LENOVO\Documents\Cours Langage C\FirstProgC\bin\Debug\FirstProgC.exe"
Bonjour UAM!
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

Éléments de base

Structuration d'un programme en C

```
#include <stdio.h>  
#include <stdlib.h>
```

```
int main()  
{  
    printf("bonjour uam!\n");  
    return 0;  
}
```

Directives du
préprocesseur

Programme principal

- Un programme écrit en C est une succession d'instructions ;
- Une instruction est une suite de mots prédéfinis, terminée par un point virgule ;
- Ces instructions sont regroupées dans différents blocs ;
- Un bloc regroupe un ensemble d'instructions entre accolades ;
- Il existe un bloc particulier : **main()**.

Éléments de base

Structuration d'un programme en C

Le préprocesseur est une étape qui s'exécute avant la compilation proprement dite. Il transforme votre code source selon des instructions qui commencent toutes par le symbole **#**. Ce préprocesseur exécute des instructions particulières appelées directives. Ces directives sont identifiées par le caractère **#** en tête.

<code>#include</code>	Inclure un fichier	<code>#include <stdio.h></code>
<code>#define</code>	Créer une macro ou une constante	<code>#define PI 3.14159</code>

Quelques exemples :

- La gestion des entrées/sorties (**stdio.h**)
- Les fonctions mathématiques (**math.h**)
- Taille des types entiers (**limits.h**)
- Limites des type réels (**float.h**)
- Traitement de chaînes de caractères (**string.h**)
- Le traitement de caractères (**ctype.h**)
- Utilitaires généraux (**stdlib.h**)
- Date et heure (**time.h**)
- Tracé de figures (**graphics.h**)

Éléments de base

Structuration d'un programme en C

La directive **#define** permet de remplacer dans le programme toutes les occurrences d'une suite de caractères par un nom de substitution.

Syntaxe de cette directive :

#define **nom_de_substitution** **suite_de_caractères**

représenter la suite de caractères *suite_de_caractères*.

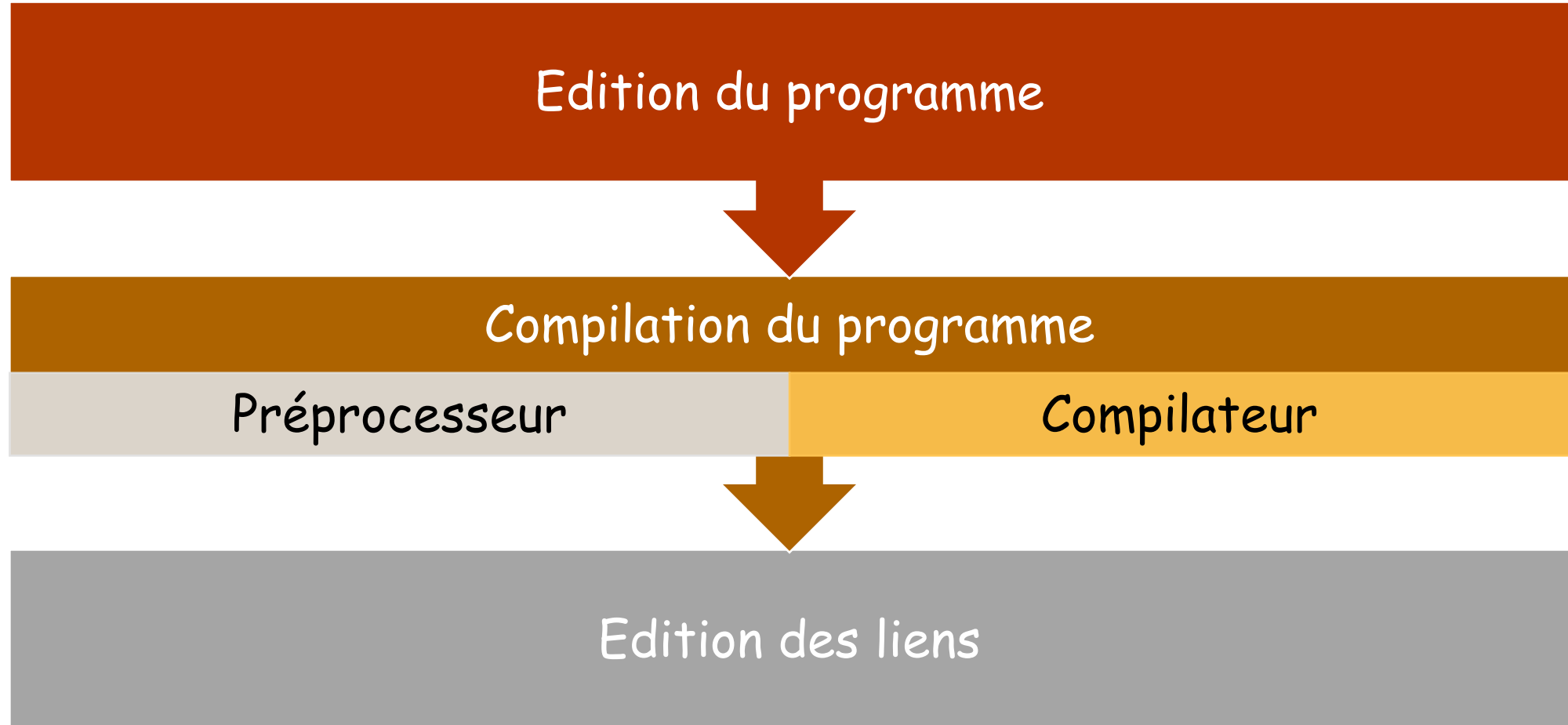
Exemple :

#define **PI** **3.14159**

Le mot PI sera utilisé pour représenter le chiffre 3.14159 dans le programme source.

Éléments de base

Compilation et Exécution d'un programme en C



Éléments de base

Commentaires

Un commentaire de code est une portion de texte explicatif intégrée au code source, ignorée par le compilateur.

En langage C, il existe deux types de commentaires :

- Commentaires par ligne en utilisant le caractère suivant : //
- Commentaires par bloc en utilisant les caractères suivants : /* */

```
1 // Ceci est un commentaire sur une seule ligne
2 int age = 25; // Un commentaire après du code
```

```
1 /* Ceci est un commentaire
2    qui s'étend sur
3    plusieurs lignes */
4
5 int x = 10; /* Commentaire à côté du code */ int y = 20;
```

Éléments de base

Variables

Ce sont des zones mémoires dont leurs valeurs peuvent changer au cours de l'exécution des instructions d'un programme en C.

Variables

Simple

Entières, Réelles,
Pointeurs...

Structurées

Tableaux,
Structures, Unions,
Listes chaînées

Éléments de base

Variables (Déclaration)

En Langage C, toute variable doit être déclarée avant utilisation

La déclaration se fait à l'aide de 2 paramètres :

<type> **<identificateur>**

Exemple :

- int i, j;
- unsigned long k;
- float x;
- double y;

Éléments de base

Variables (Identificateur ou le nom de la variable)

Un identificateur est un nom donné aux diverses composantes d'un programme : **variables, tableaux, fonctions**.

- Il est composé la plupart du temps de lettres et de chiffres.

Le 1er caractère doit obligatoirement être une lettre ou bien le caractère `_`.

- Il peut contenir jusqu'à 31 caractères minuscules et majuscules mais pas de caractères accentués (codes ASCII étendus).
- Il est d'usage de réserver les identificateurs en majuscules aux variables du préprocesseur.
- Il y a un certain nombre de **noms de variables réservés**. Leur fonction est prévue par la syntaxe du langage C et ils ne peuvent en aucun cas être utilisés dans un autre but.

Éléments de base

Variables (Identificateur ou le nom de la variable)

Les mots réservés en Langage C

auto - break - case - char - const - continue - default - do
- double - else - enum - exit - extern - float - for - goto - if
- int - long - register - return - short - signed - sizeof - static
- struct - switch - typedef - union - unsigned - void - volatile
- while

Éléments de base

Variables (Type)

En Langage C, toute variable doit être obligatoirement typée

Il y a principalement deux types de base que l'on identifie à l'aide de mots clés :

Types en C	Entier	char, short, int, long
	Flottant	float, double

Éléments de base

Variables (Type)

Le type Entier

TYPE	DESCRIPTION	TAILLE MÉMOIRE	VALEURS
int	entier standard signé	4 octets	$[-2^{31}, 2^{31} - 1]$
unsigned int	entier positif strictement	4 octets	$[0, 2^{32} - 1]$
short	entier court signé	2 octets	$[-32768, 32767]$
unsigned short	entier court non signé	2 octets	$[0, 65535]$
char	caractère signé	1 octet	$[-128, 127]$
unsigned char	caractère non signé	1 octet	$[0, 255]$
long	entier long signé	8 octets	$[-2^{63}, 2^{63} - 1]$
unsigned long	entier long non signé	8 octets	$[0, 2^{64} - 1]$

Éléments de base

Variables (Type)

Le type Réel ou Flottant

TYPE	DESCRIPTION	TAILLE MEMOIRE	MANTISSE (nombre de chiffres significatifs)	VALEURS min et max positives
float	réel standard	4 octets	6	3.4×10^{-38} à 3.4×10^{38}
double	réel double précision	8 octets	15	1.7×10^{-308} à 1.7×10^{308}
long double	réel long double précision	10 octets	19	3.4×10^{-4932} à 3.4×10^{4932}

Éléments de base

Variables (Conversion de type)

Conversion implicite

```
int main()
{
    short i = 259, j = 10;
    char ch;
    float x;

    ch = i;
    ch = 100;
    i = ch;

    x = i;
    x = 100.2;
    i = x;
}
```

- **Troncature par taille** : Quand on met un type plus grand dans un plus petit (`short → char`), on perd les bits de poids fort ;
- **Troncature par type** : Quand on met un réel dans un entier (`float → int`), on perd la partie décimale ;
- **Extension naturelle** : Quand on met un petit type dans un plus grand (`char → short`), il n'y a pas de perte.

Éléments de base

Variables (Conversion de type)

Conversion explicite

- Casting ou transtypage

```
int main()
{
    short i = 259, j = 10;
    float x;

    x = i / j;          // Division entière → x = 25.0
    x = (float) i / j;  // Division réelle → x = 25.9
}
```

Éléments de base

Opérateurs

Le langage C comporte de très nombreux opérateurs, dont :

Quelques
opérateurs en C

Arithmétique

Logique

Traitement binaire

Affectation

De Taille

Éléments de base

Opérateurs

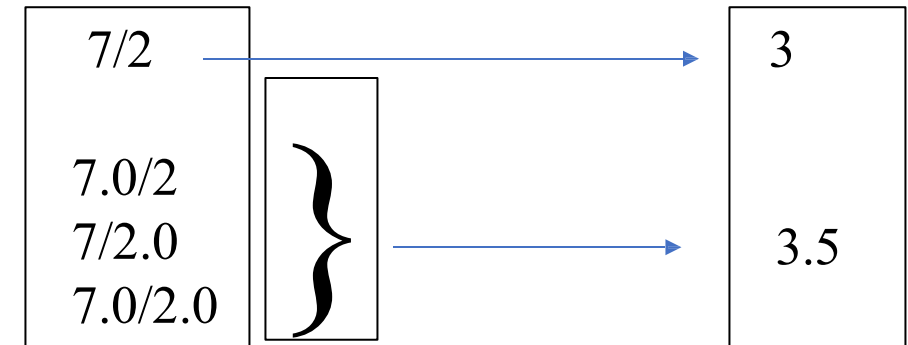
Arithmétique

Il existe 5 opérateurs arithmétiques :

- l'addition (+),
- la soustraction (-),
- la multiplication (*),
- la division (/)
- Le modulo (%) qui ne peut être utilisé qu'avec les types **entiers**.

Remarque :

Lorsque les types des deux opérandes sont différents alors il y a conversion implicite dans le type le plus fort suivant certaines règles.



Éléments de base

Opérateurs

Logique

Attention, Langage C ne possède pas de type booléen !!!

4 opérateurs relationnels

- inférieur à (<=)
- supérieur à (>)
- supérieur ou égal à (>=)
- l'opérateur de négation (!)

2 opérateurs de comparaison

- identique à (==)
- différent de (!=)

3 opérateurs de conjonction

- le ET logique (&&)
- le OU logique (||)
- le NON (!)

p	q	!p	(p) && (q)	(p) (q)
0 (F)	0 (F)	1 (V)	0	0
0 (F)	non nulle (V)	1 (V)	0	1
non nulle (V)	0 (F)	0 (F)	0	1
non nulle (V)	non nulle (V)	0 (F)	1	1

Éléments de base

Opérateurs

Traitement binaire

Les **opérateurs de traitement binaire** (ou opérateurs bit à bit) permettent de manipuler directement les bits qui composent une valeur entière. Ils sont utilisés lorsque l'on souhaite travailler au niveau le plus bas des données.

Opérateur	Nom	Rôle
&	ET binaire (AND)	Le bit résultat vaut 1 si les deux bits valent 1
	OU inclusif	OU binaire (OR)
^	OU exclusif (XOR)	Le bit résultat vaut 1 si les bits sont différents
~	Complément à 1 (NOT)	Inverse tous les bits
<<	Décalage à gauche	Décale les bits vers la gauche
>>	Décalage à droite	Décale les bits vers la droite

Éléments de base

Opérateurs

Traitement binaire sur 4 bits

Exemple avec ET binaire (&) :

Soit :

int a = 12;

int b = 10;

a & b

Nous donne 8

Exemple avec OU binaire (|) :

Soit :

int a = 12;

int b = 10;

a | b

Nous donne 14

Exemple avec XOR (^) :

Soit :

int a = 12;

int b = 10;

a ^ b

Nous donne 6

Exemple avec décalage à gauche :

Soit :

int a = 12;

a << 1

Nous donne 24

Exemple avec décalage à droite :

Soit :

int a = 12;

a >> 1

Nous donne 6

Exemple avec une inversion :

Soit :

int a = 12;

~ a

Nous donne 3

Éléments de base

Opérateurs

Affectation

Opérateurs	Opération classique	Écriture simplifiée
+=	$i = i + 20$	$i += 20$
	$i = i + k$	$i += k$
-=	$i = i - 20$	$i -= 20$
*=	$i = i * 3$	$i *= 3$
/=	$i = i / 3$	$i /= 3$
%=	$i = i \% 3$	$i \% = 3$
++i	pré-incrémentation c'est-à-dire incrémentation avant utilisation de la valeur	
i++	peut s'écrire $i = i + 1$ (post-incrémentation) incrémentation après utilisation de la valeur	
i--	peut s'écrire $i = i - 1$ (post-décrémentation)	
--i	pré-décrémentation	

Éléments de base

Opérateurs

Taille

- L'opérateur **sizeof** renvoie la taille en octets de son opérande.
- L'opérande est soit une expression soit une expression de type.
- La syntaxe est : **sizeof (expression)**.

Exemple : **sizeof(int)**

Retourne en octet le codage d'une variable de type int c'est-à-dire retourne la valeur 4.

Éléments de base

Les entrées et les sorties

`#include <stdio.h>`

- **Standard Input/Output header** (fichier d'en-tête)

// Sortie (affichage)

`printf()` // Afficher du texte formaté

`putchar()` // Afficher un caractère

`puts()` // Afficher une chaîne

// Entrée (lecture)

`scanf()` // Lire des données formatées

`getchar()` // Lire un caractère

// Gestion de fichiers

`fopen()` // Ouvrir un fichier

`fclose()` // Fermer un fichier

`fread()` // Lire un fichier

`fwrite()` // Écrire dans un fichier

Éléments de base

Les entrées et les sorties

`printf("chaîne de format", liste_d_expressions);`

d ou i	int décimal	<code>printf("%d", 42);</code>
u	unsigned int	<code>printf("%u", 42);</code>
f	float / double	<code>printf("%f", 3.14);</code>
F	float / double (inf/nan maj)	<code>printf("%F", INFINITY);</code>
e / E	Notation scientifique	<code>printf("%e", 3.14);</code> → 3.140000e+00
g / G	Plus court de f ou e	<code>printf("%g", 3.14);</code>
x / X	Hexadécimal (minuscule/maj)	<code>printf("%x", 255);</code> → ff
o	Octal	<code>printf("%o", 8);</code> → 10
s	Chaîne de caractères	<code>printf("%s", "Bonjour");</code>
c	Caractère	<code>printf("%c", 'A');</code>
p	Pointeur (adresse)	<code>printf("%p", &x);</code>

Éléments de base

Les entrées et les sorties

`scanf("chaîne de format", liste_d_adresses);`

d	int*	scanf("%d", &x);
u	unsigned int*	scanf("%u", &x);
f	float*	scanf("%f", &x);
lf	double*	scanf("%lf", &x);
Lf	long double*	scanf("%Lf", &x);
s	char* (chaîne)	scanf("%s", str);
c	char* (caractère)	scanf("%c", &c);
x	int* (hexadécimal)	scanf("%x", &x);
o	int* (octal)	scanf("%o", &x);
p	void** (pointeur)	scanf("%p", &ptr);

Éléments de base

Les entrées et les sorties (#include <stdio.h>)

```
#include <stdio.h>
```

```
int main() {
```

```
    // Exemple 1 : Alignement et largeur
```

```
    printf("|%10s|%-10s|\n", "Bonjour", "Bonjour"); // |  Bonjour|Bonjour |
```

```
    // Exemple 2 : Zéro padding pour les nombres
```

```
    printf("Numéro : %05d\n", 42); // Numéro : 00042
```

```
    // Exemple 3 : Affichage hexadécimal et octal
```

```
    int x = 255;
```

```
    printf("Décimal: %d | Hexa: %#x | Octal: %#o\n", x, x, x); // Décimal: 255 | Hexa: 0xff | Octal: 0377
```

```
    return 0;
```

```
}
```

Éléments de base

Les entrées et les sorties (#include <stdio.h>)

```
#include <stdio.h>
int main() {
    float revenue, cout_revenue, depenses;
    float resultat;
    printf("=== CALCUL DU RESULTAT D'EXPLOITATION ===\n\n");
    printf("Chiffre d'affaires : ");
    scanf("%f", &revenue);
    printf("Coût des ventes : ");
    scanf("%f", &cout_revenue);
    printf("Dépenses d'exploitation : ");
    scanf("%f", &depenses);
    resultat = revenue - cout_revenue - depenses;
    printf("\n--- RAPPORT ---\n");
    printf("Total revenue: $%.0f\n", revenue);
    printf("Total cost of revenue: $%.0f\n", cout_revenue);
    printf("Total operating expense: $%.0f\n", depenses);
    printf("Total operating income: $%.0f\n", revenue - cout_revenue);
    printf("Total operating income (loss): $%.0f\n", resultat);
    return 0;
}
```

=== CALCUL DU RESULTAT D'EXPLOITATION ===

Chiffre d'affaires : 1000

Coût des ventes : 500

Dépenses d'exploitation : 300

--- RAPPORT ---

Total revenue: \$1000

Total cost of revenue: \$500

Total operating expense: \$300

Total operating income: \$500

Total operating income (loss): \$200